

Capaian Praktikum

Setelah menyelesaikan notebook ini, mahasiswa mampu:

1. membandingkan mean, median, drop-row, constant/mode, dan missing indicator;
2. membedakan ordinal encoding yang sah dan ordinal palsu untuk kategori nominal;
3. membandingkan training pada data raw, standardization, min-max, dan robust scaling;
4. menjelaskan apa yang sebenarnya dilakukan `ColumnTransformer` dan `Pipeline` melalui implementasi manual;
5. menunjukkan mengapa leakage dapat membuat evaluasi tampak sangat baik tetapi tidak sah;
6. membandingkan majority baseline, threshold adjustment, class weighting, oversampling, dan undersampling;
7. menganalisis hasil dengan pola Fakta → Mekanisme → Makna ML → Batas Kesimpulan.

Dataset yang digunakan

File	Sifat yang sengaja dibuat
p5_imputation.csv	study_hours miring, memiliki outlier, missing lebih banyak pada kelas positif
p5_encoding.csv	program_track nominal dengan hubungan non-monotonic terhadap target
p5_encoding_ordinal_binary.csv	engagement_level benar-benar ordinal + internet_access binary
p5_scaling.csv	fitur dengan rentang sangat berbeda dan outlier study_minutes
p5_pipeline_leakage.csv	data campuran, missing, kategori, dan final_score sebagai leakage
p5_imbalance.csv	180 kelas 0 dan 20 kelas 1

0. Fungsi training manual yang dipakai berulang

Kode ini adalah pengulangan mekanisme P3. NumPy/Pandas boleh dipakai untuk array dan manipulasi data, tetapi parameter model tetap dihitung sendiri: probability, BCE, gradient, dan update weight/intercept.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def sigmoid(z):
    z = np.clip(z, -40, 40)
    return 1 / (1 + np.exp(-z))

def train_logreg_manual(X, y, lr=0.1, epochs=500, l2=0.0, sample_weight=None):
    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float)
    n, p = X.shape
    w = np.zeros(p)
    b = 0.0

    sw = np.ones(n) if sample_weight is None else np.asarray(sample_weight,
dtype=float)
```

```
    sw = sw / sw.mean()
    history = []

    for epoch in range(epochs):
         # Forward pass
        prob = sigmoid(b + X @ w)

        # Binary Cross-Entropy
        eps = 1e-12
        data_loss = np.average(
            -(y * np.log(prob + eps) + (1 - y) * np.log(1 - prob + eps)),
            weights=sw
        )

        # Objective = data loss + L2 penalty
        objective = data_loss + 0.5 * l2 * np.sum(w * w)

        # Gradient
        diff = (prob - y) * sw
        grad_w = (X.T @ diff) / n + l2 * w
        grad_b = np.mean(diff)

        # Update
        w -= lr * grad_w
        b -= lr * grad_b

        history.append(objective)

    return w, b, np.array(history)

def classification_metrics(y, pred):
    y = np.asarray(y).astype(int)
    pred = np.asarray(pred).astype(int)
    TP = np.sum((pred==1) & (y==1))
    FP = np.sum((pred==1) & (y==0))
    FN = np.sum((pred==0) & (y==1))
    TN = np.sum((pred==0) & (y==0))
    accuracy = (TP+TN)/len(y)
    precision = TP/(TP+FP) if TP+FP else 0
    recall = TP/(TP+FN) if TP+FN else 0
    f1 = 2*precision*recall/(precision+recall) if precision+recall else 0
    return TP, FP, FN, TN, accuracy, precision, recall, f1


def metric_row(label, metrics):
    TP, FP, FN, TN, accuracy, precision, recall, f1 = metrics
    return {
        "Metode": label,
        "TP": int(TP),
        "FP": int(FP),
        "FN": int(FN),
        "TN": int(TN),
        "Accuracy": float(accuracy),
        "Precision": float(precision),
        "Recall": float(recall),
        "F1": float(f1),
    }
```

```
print("Fungsi training manual siap.")
```

Checkpoint
Pastikan anda dapat menjelaskan: X dan y tetap; training mengubah w dan b. Preprocessing mengubah X sebelum training, sehingga gradient yang diterima model juga berubah.

1. Imputation Lab — Metode yang Berbeda Membuat X yang Berbeda

Data: p5_imputation.csv. Sifat penting: study_hours_week miring, ada outlier tinggi, dan beberapa missing berada pada kelas needs_support=1. Ini sengaja dibuat agar mean, median, drop row, dan missing indicator tidak menjadi pilihan yang identik.

```
df = pd.read_csv('p5_imputation.csv')
df = df.sample(frac=1, random_state=1).reset_index(drop=True)
train_imp = df.iloc[:60].copy()
val_imp   = df.iloc[60:].copy()

print("Shape train/validation:", train_imp.shape, val_imp.shape)
print("\nMissing training:")
display(train_imp.isna().sum().to_frame("missing"))

mean_fill = train_imp["study_hours_week"].mean()
median_fill = train_imp["study_hours_week"].median()

print("Mean study_hours_week   =", round(mean_fill, 3))
print("Median study_hours_week =", round(median_fill, 3))

# Visualisasi distribusi untuk memahami mengapa mean dan median dapat berbeda.
observed_study = train_imp["study_hours_week"].dropna()

plt.figure(figsize=(8, 4))
plt.hist(observed_study, bins=14, edgecolor="black")
plt.axvline(mean_fill, linestyle="--", label=f"mean={mean_fill:.2f}")
plt.axvline(median_fill, linestyle=":", label=f"median={median_fill:.2f}")
plt.xlabel("study_hours_week")
plt.ylabel("Frekuensi")
plt.title("Distribusi study_hours_week pada training")
plt.legend()
plt.show()
```

1A. Mean vs median numerik

```
mean_fill = train['study_hours_week'].mean()
median_fill = train['study_hours_week'].median()

train_mean = train.copy(); val_mean = val.copy()
train_mean['study_hours_week'] = train_mean['study_hours_week'].fillna(mean_fill)
val_mean['study_hours_week'] = val_mean['study_hours_week'].fillna(mean_fill)

train_median = train.copy(); val_median = val.copy()
train_median['study_hours_week'] =
train_median['study_hours_week'].fillna(median_fill)
val_median['study_hours_week'] = val_median['study_hours_week'].fillna(median_fill)
```

1B. Drop row

```
train_drop = train_imp.dropna(subset=["study_hours_week"]).copy()
```

```
val_drop = val_imp.dropna(subset=["study_hours_week"]).copy()

print("Train sebelum/sesudah drop:", len(train_imp), len(train_drop))
print(
    "Positive sebelum/sesudah:",
    int(train_imp["needs_support"].sum()),
    int(train_drop["needs_support"].sum())
)
```

1C. Median + missing indicator

```
train_ind = train_imp.copy()
val_ind = val_imp.copy()

train_ind["study_missing"] = train_ind["study_hours_week"].isna().astype(float)
val_ind["study_missing"] = val_ind["study_hours_week"].isna().astype(float)

train_ind["study_hours_week"] = train_ind["study_hours_week"].fillna(median_fill)
val_ind["study_hours_week"] = val_ind["study_hours_week"].fillna(median_fill)

display(train_ind.head(10))
```

1D. Standardisasi manual setelah imputasi agar perbandingan training adil

```
def fit_standard(X):
    mu = X.mean(axis=0)
    sd = X.std(axis=0)
    sd[sd == 0] = 1
    return mu, sd

def transform_standard(X, mu, sd):
    return (X - mu) / sd

def run_variant(train_df, val_df, cols, label):
    Xtr = train_df[cols].to_numpy(float)
    ytr = train_df['needs_support'].to_numpy(float)
    Xv  = val_df[cols].to_numpy(float)
    yv  = val_df['needs_support'].to_numpy(int)

    mu, sd = fit_standard(Xtr)
    Xtr = transform_standard(Xtr, mu, sd)
    Xv  = transform_standard(Xv, mu, sd)

    w, b, hist = train_logreg_manual(Xtr, ytr, lr=0.2, epochs=600, l2=0.02)
    pred = (sigmoid(b + Xv @ w) >= 0.5).astype(int)
    metrics = classification_metrics(yv, pred)
    print(label, metrics, "final J =", round(hist[-1], 4))
    return metrics, hist

imp_results = []
imp_histories = {}

metrics, hist = run_variant(
    train_drop, val_drop,
    ["prior_score", "study_hours_week"],
    "DROP"
)
imp_results.append(metric_row("DROP", metrics))
```

```
imp_histories["DROP"] = hist

metrics, hist = run_variant(
    train_mean, val_mean,
    ["prior_score", "study_hours_week"],
    "MEAN"
)
imp_results.append(metric_row("MEAN", metrics))
imp_histories["MEAN"] = hist

metrics, hist = run_variant(
    train_median, val_median,
    ["prior_score", "study_hours_week"],
    "MEDIAN"
)
imp_results.append(metric_row("MEDIAN", metrics))
imp_histories["MEDIAN"] = hist

metrics, hist = run_variant(
    train_ind, val_ind,
    ["prior_score", "study_hours_week", "study_missing"],
    "MEDIAN+INDICATOR"
)
imp_results.append(metric_row("MEDIAN+INDICATOR", metrics))
imp_histories["MEDIAN+INDICATOR"] = hist

imp_table = pd.DataFrame(imp_results)
display(imp_table.round(3))
```

```
plt.figure(figsize=(9, 5))
for label, hist in imp_histories.items():
    plt.plot(hist, label=label)

plt.xlabel("Epoch")
plt.ylabel("Objective")
plt.title("Imputation mengubah X dan dapat mengubah dinamika training")
plt.legend()
plt.grid(alpha=0.2)
plt.show()
```

Pertanyaan 1	Yang harus dicari
Mengapa mean dan median berbeda jauh?	Hubungkan dengan outlier/skew pada study_hours_week.
Mengapa drop row berisiko?	Bandingkan jumlah positive sebelum-sesudah drop.
Apa tambahan informasi dari indicator?	Dua nilai sama setelah imputasi masih bisa dibedakan karena asalnya missing/non-missing.
Bolehkah memilih metode dari test?	Tidak; gunakan validation/protokol eksperimen.

1E. Imputasi kategorikal: mode vs constant

Gunakan potongan kecil berikut agar mahasiswa melihat mekanisme sebelum masuk ke pipeline lengkap.

```
cat = pd.Series([
    "Software",
    "Software",
    "Data",
    None,
```

```
    "Software",
    "Network",
    None
])

mode_value = cat.mode(dropna=True)[0]

print("Mode =", mode_value)
print("Mode imputation   =", cat.fillna(mode_value).tolist())
print('Constant Unknown  =', cat.fillna("Unknown").tolist())
```

Analisis wajib
Mode meningkatkan frekuensi kategori mayoritas. Constant “Unknown” mempertahankan status tidak diketahui, tetapi dapat menjadi sinyal proses pengumpulan. Tidak ada metode yang selalu paling benar.

2. Encoding Lab — Uji Asumsi yang Dibawa oleh Angka

Data A: p5_encoding.csv dirancang sehingga Software dan Network sama-sama lebih berisiko daripada Data. Pola ini non-monotonic; ordinal palsu 1-2-3 akan menyulitkan satu model linear.

```
df = pd.read_csv('p5_encoding.csv')
df_enc = df_enc.sample(frac=1, random_state=2).reset_index(drop=True)
train_enc = df_enc.iloc[:80].copy()
val_enc = df_enc.iloc[80:].copy()

# attendance distandardisasi hanya dari training
mu = train_enc["attendance_pct"].mean()
sd = train_enc["attendance_pct"].std(ddof=0)

att_tr = ((train_enc["attendance_pct"] - mu) / sd).to_numpy(float)
att_val = ((val_enc["attendance_pct"] - mu) / sd).to_numpy(float)

ytr_enc = train_enc["needs_support"].to_numpy(float)
yv_enc = val_enc["needs_support"].to_numpy(int)

display(
    train_enc.groupby("program_track")["needs_support"]
    .agg(["count", "mean"])
    .rename(columns={"mean": "positive_rate"})
)
```

2A. Ordinal salah untuk nominal

```
wrong_map = {'Software':1.0, 'Data':2.0, 'Network':3.0}
Xtr_wrong = np.column_stack([att_tr, train['program_track'].map(wrong_map)])
Xv_wrong  = np.column_stack([att_val, val['program_track'].map(wrong_map)])
print(Xtr_wrong[:5])
```

2B. One-hot manual

```
categories = ["Software", "Data", "Network"]

Xtr_parts = [att_tr]
Xv_parts = [att_val]

for c in categories:
    Xtr_parts.append(
        (train_enc["program_track"] == c).astype(float).to_numpy()
```

```
    )
    Xv_parts.append(
        (val_enc["program_track"] == c).astype(float).to_numpy()
    )

Xtr_onehot = np.column_stack(Xtr_parts)
Xv_onehot = np.column_stack(Xv_parts)

print("Shape ordinal palsu:", Xtr_wrong.shape)
print("Shape one-hot      :", Xtr_onehot.shape)
print(Xtr_onehot[:5])
```

2C. Bandingkan training

```
enc_results = []
enc_histories = {}

for label, Xtr, Xv in [
    ("ORDINAL_PALSU", Xtr_wrong, Xv_wrong),
    ("ONE_HOT", Xtr_onehot, Xv_onehot),
]:
    w, b, hist = train_logreg_manual(
        Xtr, ytr_enc,
        lr=0.15,
        epochs=900,
        l2=0.01
    )

    pred = (sigmoid(b + Xv @ w) >= 0.5).astype(int)
    metrics = classification_metrics(yv_enc, pred)

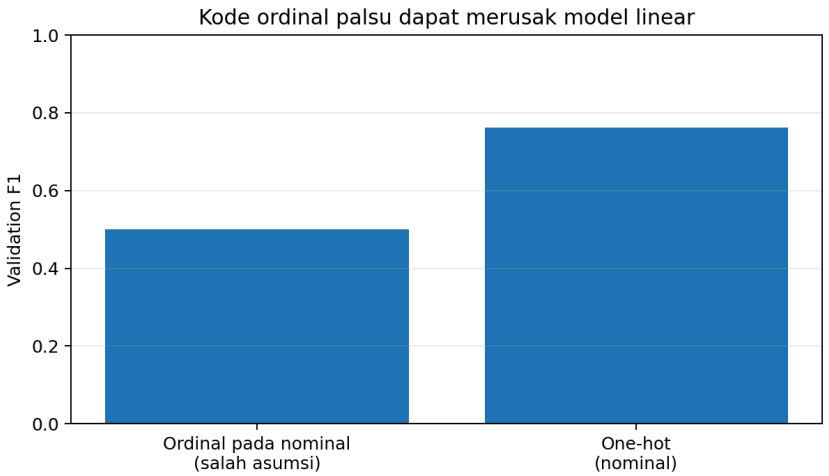
    enc_results.append(metric_row(label, metrics))
    enc_histories[label] = hist

    print(label, metrics, "J =", round(hist[-1], 4))

display(pd.DataFrame(enc_results).round(3))
```

```
plt.figure(figsize=(9, 5))
for label, hist in enc_histories.items():
    plt.plot(hist, label=label)

plt.xlabel("Epoch")
plt.ylabel("Objective")
plt.title("Ordinal palsu vs one-hot: representasi kategori mengubah training")
plt.legend()
plt.grid(alpha=0.2)
plt.show()
```



Referensi pembacaan: ordinal palsu memaksa satu arah hubungan; one-hot memberi weight terpisah untuk kategori nominal.

2D. Binary dan ordinal yang benar

```
df_enc2 = pd.read_csv("p5_encoding_ordinal_binary.csv")

engagement_map = {
    "Low": 0.0,
    "Medium": 1.0,
    "High": 2.0
}

internet_map = {
    "No": 0.0,
    "Yes": 1.0
}

df_enc2["engagement_code"] = df_enc2["engagement_level"].map(engagement_map)
df_enc2["internet_code"] = df_enc2["internet_access"].map(internet_map)

display(
    df_enc2[
        ["engagement_level", "engagement_code",
         "internet_access", "internet_code"]
    ].head(10)
)
```

Pertanyaan 2
Mengapa engagement_level boleh ordinal tetapi program_track tidak? Jawaban harus menyebut struktur domain, bukan karena keduanya sama-sama berupa teks.

3. Scaling Lab — Learning Rate Sama, Representasi Berbeda

Data: p5_scaling.csv. attendance_pct, practice_hours, dan study_minutes memiliki rentang berbeda; study_minutes juga mengandung outlier. Tujuan: menggunakan training manual yang sama dan learning rate sama untuk melihat dampak skala.

```
df = pd.read_csv('p5_scaling.csv')
df_scale = df.sample(frac=1, random_state=3).reset_index(drop=True)

train_scale = df_scale.iloc[:100].copy()
val_scale = df_scale.iloc[100:].copy()

scale_cols = [
```

```
    "attendance_pct",
    "practice_hours",
    "study_minutes"
]

Xtr_raw = train_scale[scale_cols].to_numpy(float)
Xv_raw = val_scale[scale_cols].to_numpy(float)

ytr_scale = train_scale["needs_support"].to_numpy(float)
yv_scale = val_scale["needs_support"].to_numpy(int)

display(train_scale[scale_cols].describe().T)
```

3A. Fit/transform scaling manual

```
def fit_scaler(X, method):
    if method == "none":
        return None

    if method == "standard":
        mu = X.mean(axis=0)
        sd = X.std(axis=0)
        sd[sd == 0] = 1
        return mu, sd

    if method == "minmax":
        mn = X.min(axis=0)
        rg = X.max(axis=0) - mn
        rg[rg == 0] = 1
        return mn, rg

    if method == "robust":
        med = np.median(X, axis=0)
        q1 = np.percentile(X, 25, axis=0)
        q3 = np.percentile(X, 75, axis=0)
        iqr = q3 - q1
        iqr[iqr == 0] = 1
        return med, iqr

    raise ValueError("method tidak dikenal")


def transform_scaler(X, method, state):
    if method == "none":
        return X.copy()

    center, scale = state
    return (X - center) / scale
```

3B. Bandingkan loss curve dan validation

```
scale_results = []
scale_histories = {}

plt.figure(figsize=(9, 5))

for method in ["none", "standard", "minmax", "robust"]:
    state = fit_scaler(Xtr_raw, method)
```

```
    Xtr = transform_scaler(Xtr_raw, method, state)
    Xv = transform_scaler(Xv_raw, method, state)

    w, b, hist = train_logreg_manual(
        Xtr, ytr_scale,
        lr=0.12,
        epochs=300,
        l2=0.01
    )

    pred = (sigmoid(b + Xv @ w) >= 0.5).astype(int)
    metrics = classification_metrics(yv_scale, pred)

    scale_results.append(metric_row(method, metrics))
    scale_histories[method] = hist

    plt.plot(hist, label=method)

    print(method, metrics, "final J =", round(hist[-1], 4))

plt.ylim(0, 5)
plt.xlabel("Epoch")
plt.ylabel("Objective")
plt.title("Learning rate sama, representasi X berbeda")
plt.legend()
plt.grid(alpha=0.2)
plt.show()

display(pd.DataFrame(scale_results).round(3))
```

Pertanyaan 3 (tunjukkan bukti nya)	Makna
Raw objective melonjak/berosilasi	gradient besar karena skala fitur besar; learning rate yang sama menjadi tidak cocok
Standardization turun stabil	feature berpusat dan skala lebih sebanding
Min-max lambat pada data ini	outlier study_minutes menentukan max dan memampatkan mayoritas nilai
Robust stabil	median/IQR lebih tahan terhadap outlier

Batas kesimpulan
Jangan menulis “robust selalu terbaik”. Kesimpulan yang sah adalah: pada dataset ini, dengan outlier yang sengaja ada dan learning rate yang sama, representasi robust/standard memberi dinamika training yang lebih stabil.

4. Manual ColumnTransformer + Pipeline — Tanpa scikit-learn

Data: p5_pipeline_leakage.csv. Kita akan membangun “state preprocessing” sendiri agar mahasiswa melihat apa yang sebenarnya dilakukan ColumnTransformer/Pipeline: setiap cabang mempelajari state dari training, lalu state yang sama dipakai pada validation/test.

```
df_pipe = pd.read_csv("p5_pipeline_leakage.csv")
df_pipe = df_pipe.sample(frac=1, random_state=8).reset_index(drop=True)

train_pipe = df_pipe.iloc[:140].copy()
val_pipe = df_pipe.iloc[140:180].copy()
test_pipe = df_pipe.iloc[180:].copy()
```

```
print(train_pipe.shape, val_pipe.shape, test_pipe.shape)
display(train_pipe.head())


def fit_preprocessor(train_df, include_leak=False):
    state = {}

    num_cols = [
        "attendance_pct",
        "prior_score",
        "study_hours_week"
    ]

    if include_leak:
        num_cols = num_cols + ["final_score"]

    state["num_cols"] = num_cols

    # Imputation state
    state["median"] = {
        c: train_df[c].median()
        for c in num_cols
    }

    X_num = np.column_stack([
        train_df[c].fillna(state["median"][c]).to_numpy(float)
        for c in num_cols
    ])

    # Scaling state
    state["mean"] = X_num.mean(axis=0)
    state["sd"] = X_num.std(axis=0)
    state["sd"][state["sd"] == 0] = 1

    # Nominal category state
    state["program_mode"] = train_df["program_track"].mode(dropna=True)[0]
    prog = train_df["program_track"].fillna(state["program_mode"])
    state["program_categories"] = sorted(prog.unique().tolist())

    # Ordinal/binary state
    state["engagement_mode"] = train_df["engagement_level"].mode(dropna=True)[0]
    state["engagement_map"] = {
        "Low": 0.0,
        "Medium": 1.0,
        "High": 2.0
    }
    state["internet_map"] = {
        "No": 0.0,
        "Yes": 1.0
    }

    return state


def transform_preprocessor(df, state):
    # Cabang numerik
    X_num = np.column_stack([
```

```
        df[c].fillna(state["median"][c]).to_numpy(float)
        for c in state["num_cols"]
    ])
    X_num = (X_num - state["mean"]) / state["sd"]

    # Cabang nominal: one-hot
    prog = df["program_track"].fillna(state["program_mode"])
    onehot = np.column_stack([
        (prog == cat).astype(float).to_numpy()
        for cat in state["program_categories"]
    ])

    # Cabang ordinal
    eng = df["engagement_level"].fillna(state["engagement_mode"])
    eng = (
        eng.map(state["engagement_map"])
        .fillna(1.0)
        .to_numpy(float)
        .reshape(-1, 1)
    )

    # Cabang binary
    internet = (
        df["internet_access"]
        .map(state["internet_map"])
        .fillna(0.5)
        .to_numpy(float)
        .reshape(-1, 1)
    )

    return np.column_stack([
        X_num,
        onehot,
        eng,
        internet
    ])


state_legal = fit_preprocessor(train_pipe, include_leak=False)

X_train_pipe = transform_preprocessor(train_pipe, state_legal)
X_val_pipe = transform_preprocessor(val_pipe, state_legal)
X_test_pipe = transform_preprocessor(test_pipe, state_legal)

y_train_pipe = train_pipe["needs_support"].to_numpy(float)
y_val_pipe = val_pipe["needs_support"].to_numpy(int)
y_test_pipe = test_pipe["needs_support"].to_numpy(int)

print("Shape sesudah preprocessing:")
print(X_train_pipe.shape, X_val_pipe.shape, X_test_pipe.shape)

print("\nState yang dipelajari dari TRAINING:")
print("Median numerik     :", state_legal["median"])
print("Mean numerik       :", np.round(state_legal["mean"], 3))
print("SD numerik         :", np.round(state_legal["sd"], 3))
print("Program mode       :", state_legal["program_mode"])
print("Program categories :", state_legal["program_categories"])
print("Engagement mode    :", state_legal["engagement_mode"])
```

Pertanyaan 4
Tuliskan state mana yang fit dari training dan mana yang hanya merupakan aturan domain.
Jika state dihitung ulang pada validation/test, apa yang salah dari prosedurnya?.

5. Leakage Lab — Skor Sangat Bagus Bisa Menjadi Bukti Kesalahan Eksperimen

Kita bandingkan dua pipeline yang identik.

- LEGAL: hanya fitur yang tersedia sebelum outcome;
- LEAKAGE: memasukkan final_score, padahal target needs_support berkaitan langsung dengan outcome akhir.

.

```
def train_and_eval_pipeline(include_leak):
    state = fit_preprocessor(
        train_pipe,
        include_leak=include_leak
    )

    Xtr = transform_preprocessor(train_pipe, state)
    Xv = transform_preprocessor(val_pipe, state)

    ytr = train_pipe["needs_support"].to_numpy(float)
    yv = val_pipe["needs_support"].to_numpy(int)

    w, b, hist = train_logreg_manual(
        Xtr, ytr,
        lr=0.15,
        epochs=800,
        l2=0.02
    )

    pred = (sigmoid(b + Xv @ w) >= 0.5).astype(int)
    metrics = classification_metrics(yv, pred)

    return metrics, hist

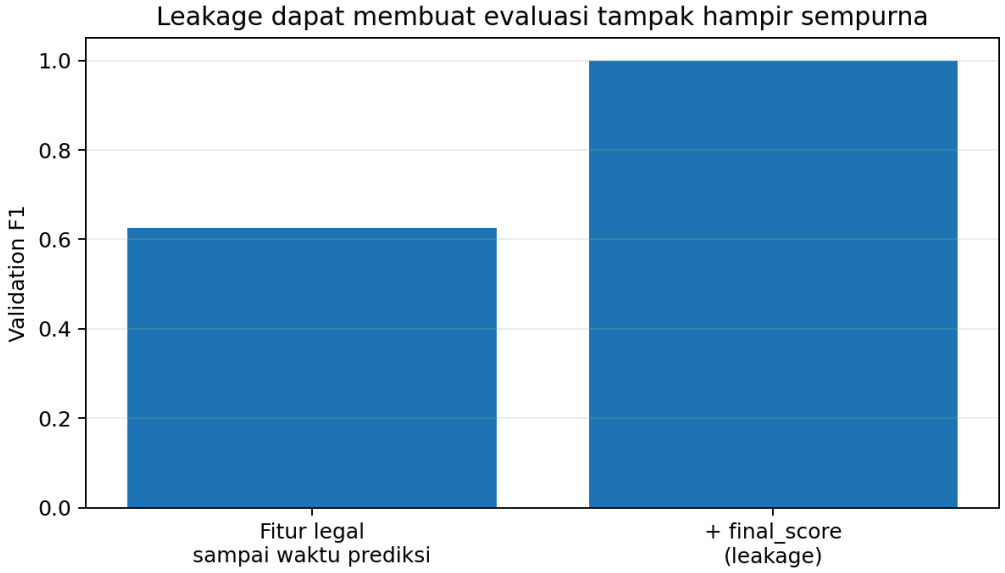
legal_result, h_legal = train_and_eval_pipeline(False)
leak_result, h_leak = train_and_eval_pipeline(True)

leak_table = pd.DataFrame([
    metric_row("LEGAL", legal_result),
    metric_row("LEAKAGE", leak_result)
])

display(leak_table.round(3))
```

```
plt.figure(figsize=(9, 5))
plt.plot(h_legal, label="Legal features")
plt.plot(h_leak, label="Dengan final_score (leakage)")
plt.xlabel("Epoch")
plt.ylabel("Objective")
plt.title("Leakage dapat membuat training/evaluasi tampak sangat mudah")
plt.legend()
plt.grid(alpha=0.2)
```

```
plt.show()
```



Referensi: pipeline leakage dapat tampak "hebat"; mahasiswa harus menolak hasilnya berdasarkan waktu prediksi, bukan menerima karena F1 tinggi.

Tambahan: train-test contamination

```
# SALAH: state preprocessing dihitung sebelum split
median_all = df_pipe["attendance_pct"].median()
mean_all = df_pipe["attendance_pct"].fillna(median_all).mean()

# BENAR: state hanya dari training
median_train = train_pipe["attendance_pct"].median()
mean_train = train_pipe["attendance_pct"].fillna(median_train).mean()

print("median all vs train =", median_all, median_train)
print("mean all vs train   =", mean_all, mean_train)
```

Pertanyaan 5
Walaupun perbedaan statistik tampak kecil, mengapa prosedur "fit pada seluruh data" tetap tidak sah? Jawaban harus berbicara tentang batas informasi, bukan besar selisih angka.

6. Imbalance Lab — Bandingkan Lima Strategi pada Data 90:10

Data: p5_imbalance.csv berisi 180 negative dan 20 positive. Gunakan satu split tetap. Fokusnya bukan mencari satu pemenang universal, tetapi melihat mekanisme yang berbeda.

```
df_imb = pd.read_csv("p5_imbalance.csv")
df_imb = df_imb.sample(frac=1, random_state=5).reset_index(drop=True)

train_imb = df_imb.iloc[:150].copy()
val_imb = df_imb.iloc[150:].copy()

imb_cols = ["feature1", "feature2"]

Xtr_imb = train_imb[imb_cols].to_numpy(float)
Xv_imb = val_imb[imb_cols].to_numpy(float)

ytr_imb = train_imb["needs_support"].to_numpy(float)
yv_imb = val_imb["needs_support"].to_numpy(int)
```

```
mu = Xtr_imb.mean(axis=0)
sd = Xtr_imb.std(axis=0)
sd[sd == 0] = 1

Xtr_imb = (Xtr_imb - mu) / sd
Xv_imb = (Xv_imb - mu) / sd

print("Class training   =", np.bincount(ytr_imb.astype(int)))
print("Class validation =", np.bincount(yv_imb.astype(int)))
```

6A. Majority baseline

```
imb_results = []

baseline = np.zeros_like(yv_imb)
baseline_metrics = classification_metrics(yv_imb, baseline)

imb_results.append(metric_row("Majority baseline", baseline_metrics))

print("BASELINE:", baseline_metrics)
```

6B. Training biasa + threshold adjustment

```
w_imb, b_imb, h_imb = train_logreg_manual(
    Xtr_imb, ytr_imb,
    lr=0.15,
    epochs=600,
    l2=0.01
)

p_imb = sigmoid(b_imb + Xv_imb @ w_imb)

threshold_results = []

for t in [0.50, 0.30, 0.25, 0.20]:
    pred = (p_imb >= t).astype(int)
    metrics = classification_metrics(yv_imb, pred)

    threshold_results.append(
        {"threshold": t, **metric_row(f"Threshold {t:.2f}", metrics)}
    )

    print("threshold", t, metrics)

display(pd.DataFrame(threshold_results).round(3))
```

```
w_imb, b_imb, h_imb = train_logreg_manual(
    Xtr_imb, ytr_imb,
    lr=0.15,
    epochs=600,
    l2=0.01
)

p_imb = sigmoid(b_imb + Xv_imb @ w_imb)

threshold_results = []
```

```
for t in [0.50, 0.30, 0.25, 0.20]:
    pred = (p_imb >= t).astype(int)
    metrics = classification_metrics(yv_imb, pred)

    threshold_results.append(
        {"threshold": t, **metric_row(f"Threshold {t:.2f}", metrics)}
    )

    print("threshold", t, metrics)

display(pd.DataFrame(threshold_results).round(3))
```

6C. Class weighting manual

```
n0 = np.sum(ytr_imb == 0)
n1 = np.sum(ytr_imb == 1)

weights = np.where(
    ytr_imb == 1,
    len(ytr_imb) / (2 * n1),
    len(ytr_imb) / (2 * n0)
)

w_cw, b_cw, h_cw = train_logreg_manual(
    Xtr_imb, ytr_imb,
    lr=0.15,
    epochs=600,
    l2=0.01,
    sample_weight=weights
)

pred_cw = (sigmoid(b_cw + Xv_imb @ w_cw) >= 0.5).astype(int)
cw_metrics = classification_metrics(yv_imb, pred_cw)

imb_results.append(metric_row("Class weight", cw_metrics))
print("CLASS WEIGHT:", cw_metrics)
```

6D. Random oversampling manual

```
rng = np.random.default_rng(1)

pos = np.where(ytr_imb == 1)[0]
neg = np.where(ytr_imb == 0)[0]

pos_os = rng.choice(
    pos,
    size=len(neg),
    replace=True
)

idx_os = np.concatenate([neg, pos_os])
rng.shuffle(idx_os)

w_os, b_os, h_os = train_logreg_manual(
    Xtr_imb[idx_os],
    ytr_imb[idx_os],
    lr=0.15,
    epochs=600,
```

```
    l2=0.01
)

pred_os = (sigmoid(b_os + Xv_imb @ w_os) >= 0.5).astype(int)
os_metrics = classification_metrics(yv_imb, pred_os)

imb_results.append(metric_row("Oversampling", os_metrics))
print("OVERSAMPLING:", os_metrics)
print("Ukuran training setelah oversampling:", len(idx_os))
```

6E. Random undersampling manual

```
neg_us = rng.choice(
    neg,
    size=len(pos),
    replace=False
)

idx_us = np.concatenate([pos, neg_us])
rng.shuffle(idx_us)

w_us, b_us, h_us = train_logreg_manual(
    Xtr_imb[idx_us],
    ytr_imb[idx_us],
    lr=0.15,
    epochs=600,
    l2=0.01
)

pred_us = (sigmoid(b_us + Xv_imb @ w_us) >= 0.5).astype(int)
us_metrics = classification_metrics(yv_imb, pred_us)

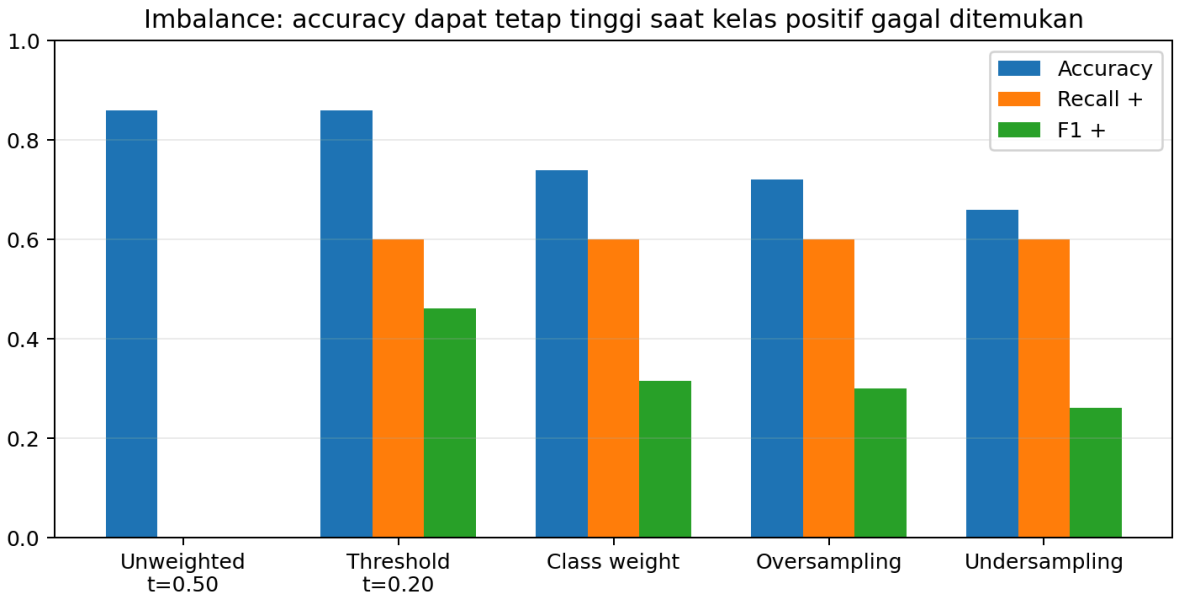
imb_results.append(metric_row("Undersampling", us_metrics))
print("UNDERSAMPLING:", us_metrics)
print("Ukuran training setelah undersampling:", len(idx_us))
```

6F. Sintesis imbalance

```
imb_table = pd.DataFrame(imb_results)
display(imb_table.round(3))


plot_df = imb_table.set_index("Metode")[["Accuracy", "Precision", "Recall", "F1"]]

ax = plot_df.plot(kind="bar", figsize=(11, 5))
ax.set_ylim(0, 1.05)
ax.set_ylabel("Metric")
ax.set_title("Strategi imbalance menghasilkan trade-off metrik yang berbeda")
plt.xticks(rotation=25, ha="right")
plt.grid(axis="y", alpha=0.2)
plt.show()
```



Contoh hasil dataset latihan: strategi yang meningkatkan recall minoritas dapat menurunkan accuracy/precision. Tugas mahasiswa adalah menjelaskan mekanismenya.

Strategi	Apa yang benar-benar berubah?	Pertanyaan analisis 6
Threshold	aturan probability → kelas	mengapa weight sama tetapi TP/FP/FN berubah?
Class weight	kontribusi tiap observasi ke loss/gradient	mengapa kesalahan positive menjadi lebih mahal?
Oversampling	frekuensi observasi positive di training	mengapa gradient lebih sering melihat positive yang sama?
Undersampling	jumlah majority yang dipakai training	informasi apa yang mungkin hilang?

7. Sintesis: Cara Menulis Analisis, Bukan Hanya Output

Lapisan	Pertanyaan 7	Perjelas jawaban ini dan tunjukkan dari hasil
Fakta	Apa angka/grafik yang terlihat?	Recall naik dari 0 menjadi 0.60 setelah class weighting.
Mekanisme	Apa yang berubah di proses?	Positive mendapat bobot loss lebih besar sehingga gradient lebih kuat memperbaiki kesalahan positive.
Makna ML	Konsep apa yang dibuktikan?	Imbalance dapat memerlukan objective yang cost-sensitive; accuracy saja tidak cukup.
Batas kesimpulan	Apa yang belum boleh diklaim?	Belum berarti class weighting terbaik untuk deployment; perlu validation, biaya FP/FN, dan test final.

Template analisis

"Saya mengamati ____. Hal ini terjadi karena preprocessing/strategi ____ mengubah ____. Secara konsep ML, hasil ini menunjukkan ____. Namun hasil ini belum membuktikan ____ karena ____."

8. Tugas Pengumpulan

1. Notebook berjalan dari atas ke bawah tanpa library training/model.
2. Tabel perbandingan imputation: jumlah data, nilai fill, final objective, precision/recall/F1 validation.
3. Perbandingan ordinal palsu vs one-hot dan penjelasan asumsi encoding.
4. Grafik loss raw/standard/min-max/robust dan interpretasi gradient descent.
5. Daftar state manual ColumnTransformer/Pipeline yang fit pada training.
6. Perbandingan fitur legal vs leakage dan alasan mengapa skor leakage harus ditolak.
7. Tabel imbalance: majority baseline, threshold, class weight, oversampling, undersampling.
8. Analisis akhir 400–600 kata dengan pola Fakta → Mekanisme → Makna ML → Batas Kesimpulan.